CodePilot RAG

Autonomous AI Coding Assistant & Context Engine

Technical Documentation

Surface	URL
Web application	`https://frontend-theta-olive-55.vercel.app/`
Backend API	`https://codepilot-snj4.onrender.com`
Interactive API docs	`https://codepilot-snj4.onrender.com/docs`
Health check	`https://codepilot-snj4.onrender.com/health`

A multi-agent Retrieval-Augmented Generation system that ingests a software repository, indexes it semantically, and answers questions about it with citations to the exact files consulted. Beyond question answering it diagnoses bugs, reviews code quality, explains architecture, and generates unified git diffs that can be applied in a single click.

Layer	Technology
Frontend	Next.js 14 (App Router), React 18, TypeScript, Tailwind CSS
Backend	FastAPI, Uvicorn, Pydantic Settings
Agent orchestration	LangGraph, LangChain
Language model	Google Gemini (models/gemini-2.5-flash)
Embeddings	models/gemini-embedding-2 (3072 dimensions)
Vector store	ChromaDB (embedded PersistentClient)
Database	MongoDB with Beanie ODM
Hosting	Vercel (web) / Render (API) / MongoDB Atlas (data)

1. System Architecture

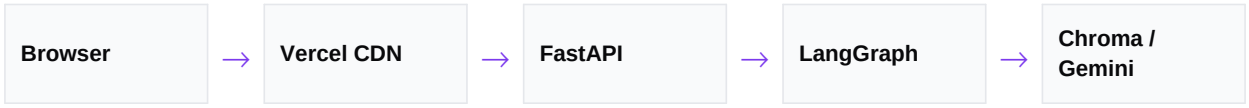
The web application and the API are deployed and scaled independently. The browser calls the API directly with no intermediate proxy, which is why cross-origin configuration is a load-bearing part of the deployment rather than an afterthought.

Tier	Responsibility	Persistence
Browser	Renders the UI; issues cross-origin calls to the API.	None
Vercel	Serves the static Next.js bundle. The API base URL is compiled into the bundle at build time.	None
Render	FastAPI process, LangGraph agent, embedded Chroma client.	Disk mount at /app/data
MongoDB Atlas	Repository metadata, threads, messages, background jobs.	Managed
Google Gemini	Chat completions and embedding vectors.	External

Storage split and why it matters

Repository metadata lives in MongoDB while the vectors and uploaded archives live on the API container's local disk. These two stores can drift apart. If the container restarts without a mounted persistent disk, the vector index and uploaded archives are erased while MongoDB continues to report the repository as ready.

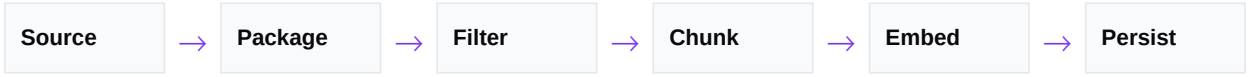
The observable symptom is subtle and dangerous: the dashboard looks healthy, the file browser returns an error, and the agent answers questions from an empty context. Section 7 describes the guards that now make this state visible instead of silent.



Request path. MongoDB Atlas is consulted by FastAPI for metadata and conversation history at the start and end of every chat request.

2. Ingestion Pipeline

Ingestion is asynchronous. The API validates the source, registers a background job, and returns HTTP 202 with a job identifier immediately. The browser then polls the job endpoint every 1.5 seconds to drive a progress indicator, so a large repository never blocks the request thread.



Stage	Detail
Source	One of: an absolute directory path on the server, a public GitHub URL cloned over HTTPS, or an uploaded ZIP archive.
Package	The tree is walked and compressed to a ZIP stored under UPLOAD_DIR. The archive is retained so patches can later be applied to it.
Filter	Excluded directories: .git, node_modules, venv, .next, __pycache__, .pytest_cache. Binary and lock files are skipped.

Stage	Detail
Chunk	RecursiveCharacterTextSplitter, 1000 characters with 200 characters of overlap, capped at 50 chunks per file.
Embed	Submitted to Gemini in batches of 5 with exponential backoff, because free-tier embedding quota rate-limits aggressively.
Persist	Vectors are written to a per-repository Chroma collection named repo-{repo_id}; counts and languages are recorded in MongoDB.

3. Agent Workflow

The core of the system is a LangGraph state machine of six nodes joined by five conditional routers. A single generic prompt gives mediocre answers to every kind of question; routing lets each class of request follow a pipeline suited to it.

3.1 Nodes

Node	Purpose	Token cap
classify_request	Resolves an 'auto' request into a concrete mode. Falls back to keyword matching if the model returns malformed JSON.	128
retrieve_context	Embeds the query and pulls the top-k chunks, discarding anything below the relevance floor.	n/a
plan_solution	Produces an implementation plan. Debug and patch modes only.	512
generate_answer	Produces the prose answer for question, review and architecture modes.	2048
generate_patch	Produces a unified diff inside a fenced block.	2048
verify_output	Checks the generated patch against the retrieved context.	512

3.2 Routing rules

Transcribed directly from the router functions in app/agent/graph.py.

Router	Condition	Destination
route_start	mode == 'auto'	classify_request
	otherwise	retrieve_context
route_by_mode	mode is a recognised mode	retrieve_context
	otherwise	END
route_after_retrieve	mode in {debug, patch}	plan_solution
	otherwise	generate_answer
route_after_planning	mode == 'patch'	generate_patch
	otherwise	generate_answer
route_after_generation	mode == 'patch'	verify_output
	otherwise	END

Only patch mode reaches the verification node. Reviews and architecture answers return directly, which is why they complete noticeably faster.

3.3 Router implementation

```
def route_start(state: AgentState) -> Literal["classify_request", "retrieve_context"]:
    """Bypasses classification if mode is already set explicitly."""
    if state.get("mode") == "auto":
        return "classify_request"
    return "retrieve_context"


def route_after_retrieve(state: AgentState) -> Literal["plan_solution", "generate_answer"]:
    """Routes debug and patch modes to planning first; others to generation."""
    mode = state.get("mode", "question")
    if mode in ["debug", "patch"]:
        return "plan_solution"
    return "generate_answer"
```

4. Retrieval and Grounding

This section covers the part of the system that most determines whether the output can be trusted. A retrieval-augmented generator is only as honest as its willingness to admit that retrieval failed.

4.1 Scored retrieval

An unfiltered top-k search always returns k chunks, even when every one of them is irrelevant. Those chunks then reach the model formatted exactly like legitimate grounding. Filtering on relevance score means an off-topic question ends up with zero chunks, which the generation nodes treat as a refusal condition.

```
results = await vector_service.similarity_search_with_score(
    repo_id=repo_id,
    query=search_query,
    k=settings.retrieval_k,
)

chunks = []
for doc, score in results:
    score = float(score)
    if score < settings.retrieval_min_score:
        discarded += 1
        continue
    chunks.append({
        "page_content": doc.page_content,
        "file_path": doc.metadata.get("file_path", "unknown"),
        "score": score,
    })
```

4.2 Calibrating the relevance floor

The threshold was measured rather than guessed. Against the gemini-embedding-2 model on a small web project, on-topic and off-topic queries separate cleanly:

Query type	Observed chunk scores	Outcome at floor = 0.35
On topic	0.4588 - 0.5200	Answered with 4 citations
Off topic	0.2164 - 0.2361	Refused, no model call

A floor of 0.35 sits inside that gap. This is calibrated on a limited sample and should be treated as a starting point: if legitimate questions begin to be refused, lower it. Changing the embedding model invalidates the number entirely.

4.3 The grounding gate

When no chunk clears the floor, the generation nodes return a fixed explanatory response without calling the language model at all. This is both correct and free: an empty context block does not stop a model from producing a fluent, confident, entirely invented answer, and such an answer is indistinguishable from a good one at a glance.

```
if not state.get("retrieved_chunks"):
    logger.warning(
        "No context for repo=%s mode=%s - refusing to generate",
        state.get("repo_id"), state.get("mode"),
    )
    return {"response": NO_CONTEXT_RESPONSE, "citations": [], "verified": True}
```

The equivalent gate on the patch node matters most. A diff generated without real file context is a patch against imagined code: it either fails to apply or, worse, applies in the wrong place.

4.4 Citations

Citations are built from what retrieval actually returned, ranked by score and de-duplicated by file. An earlier implementation attached a citation only when the model happened to repeat a file path verbatim in its prose, so any summary-style answer silently lost all of its sources even though retrieval had worked correctly.

```
def build_citations(chunks: list[dict[str, Any]]) -> list[dict[str, Any]]:
    citations, seen = [], set()
    for chunk in sorted(chunks, key=lambda c: c.get("score", 0.0), reverse=True):
        file_path = chunk.get("file_path", "unknown")
        if file_path in seen:
            continue
        seen.add(file_path)
        citations.append({
            "file_path": file_path,
            "snippet": chunk.get("page_content", "")[:200] + "...",
            "score": round(float(chunk.get("score", 0.0)), 4),
        })
        if len(citations) >= settings.max_citations:
            break
    return citations
```

5. Operational Modes

Mode	Behaviour	Plans	Verifies
auto	Classifies the request and dispatches to one of the modes below.	-	-
question	Answers questions about the indexed codebase.	No	No
debug	Plans first, then traces a failure to its root cause.	Yes	No
patch	Plans, emits a unified diff, then verifies it.	Yes	Yes
review	Code quality report with severity indicators.	No	No
architecture	Explains layers, flows and module boundaries.	No	No

5.1 Abuse and cost controls

Control	Value	Implementation
Chat rate limit	10 requests / minute / IP	Sliding window guarded by asyncio.Lock; returns HTTP 429.
Classification cap	128 tokens	Per-call max_output_tokens.
Planning cap	512 tokens	Per-call max_output_tokens.
Generation cap	2048 tokens	Applies to answers and patches.
Request timeout	45 seconds	Hard per-call cap with 2 retries.
Upload ceiling	100 MB	Configurable via MAX_UPLOAD_SIZE_MB.

6. API Reference

Base URL: https://codepilot-snj4.onrender.com

6.1 System

Method	Endpoint	Description
GET	/	Service metadata
GET	/health	MongoDB and ChromaDB connectivity
GET	/docs	Interactive OpenAPI documentation

6.2 Repositories

Method	Endpoint	Description
POST	/repos/upload	Ingest a ZIP upload or clone a public GitHub URL
POST	/repos/local	Ingest an absolute server-side directory path
GET	/repos	List repositories, reconciled against the live index

Method	Endpoint	Description
DELETE	/repos/{repo_id}	Delete metadata, archive and vector collection
GET	/repos/{repo_id}/files	List indexed file paths
GET	/repos/{repo_id}/files/content	Read a single file by path
GET	/repos/{repo_id}/download	Download the working archive

6.3 Chat, threads, patches and jobs

Method	Endpoint	Description
POST	/chat	Run the agent. Body: query, repo_id, thread_id, mode
GET	/threads/{repo_id}	List conversation threads for a repository
GET	/threads/{thread_id}/messages	Full message history
DELETE	/threads/{thread_id}	Delete a thread
POST	/patch/apply	Apply a unified diff, then re-index
GET	/jobs/{job_id}	Poll background job progress

6.4 Example

```
curl -X POST https://codepilot-snj4.onrender.com/chat \
  -H "Content-Type: application/json" \
  -d '{
    "query": "Explain the project structure and main entry point.",
    "repo_id": "your-repo-id",
    "mode": "question"
  }'
```

7. Production Hardening

The following changes separate a working prototype from a system that can be exposed to the internet. Each was driven by an observed failure rather than a checklist.

7.1 Correctness

Problem	Resolution
Generation proceeded with an empty context block, producing confident fabrications indistinguishable from grounded answers.	Grounding gate on every generation node; the model is not called at all when retrieval returns nothing.
Top-k returned k chunks regardless of relevance, so off-topic questions still received formatted 'context'.	Relevance floor applied to scored retrieval, calibrated against measured score distributions.
Citations only attached when the model echoed a file path verbatim.	Citations built from retrieved chunks, ranked by score and de-duplicated.
Repositories reported 'ready' with a populated chunk count while the underlying vector collection was empty.	The repository listing is reconciled against the live index and reports 'error' when the two disagree.
A missing or corrupt collection raised a 500.	Retrieval failures degrade into the insufficient-context response.

7.2 Security

Problem	Resolution
Every endpoint was unauthenticated: anyone could ingest repositories, delete indexes, or exhaust language-model quota.	Opt-in API key on all mutating endpoints, compared in constant time. Unset means disabled, so local development is unchanged.
The local-path ingestion endpoint reads an arbitrary directory from the server filesystem - a directory-disclosure primitive when hosted.	Gated behind ALLOW_LOCAL_INGEST and optionally confined to a single root; paths are resolved before the containment check so traversal cannot escape.
Preview deployments could not be allowed through CORS without opening it to every site on the hosting provider's domain.	A narrowly scoped origin regular expression, matched in full, restricted to the project's own hostname pattern.

7.3 Authentication implementation

```
def _verify(provided: str | None) -> None:
    expected = settings.api_key

    # Auth disabled - nothing configured, so every request is allowed.
    if not expected:
        return

    if not provided:
        raise HTTPException(status_code=401, detail="Missing API key.")

    # compare_digest avoids leaking key length/prefix through timing.
    if not secrets.compare_digest(provided, expected):
        raise HTTPException(status_code=401, detail="Invalid API key.")
```

7.4 Verified behaviour

Scenario	Result
Mutating endpoint, no key	401 Unauthorized
Mutating endpoint, wrong key	401 Unauthorized
Mutating endpoint, correct key	Passes to handler
Read endpoints, no key	200 OK, reads stay public
Local ingest while disabled	403 with actionable message
On-topic question	4 citations, answered in 6.8 s
Off-topic question	Refused in 0.49 s, no model call
Empty vector index	Refused in 2.0 s, no model call

8. Configuration Reference

Loaded by Pydantic Settings from ../.env then .env, relative to the backend working directory. Environment variables take precedence.

8.1 Required

Variable	Default	Notes
GOOGLE_API_KEY	(empty)	Gemini key used for chat and embeddings
MONGODB_URL	mongodb://localhost:27017	Atlas deployments use a mongodb+srv:// connection string

8.2 Retrieval

Variable	Default	Notes
RETRIEVAL_K	8	Chunks requested per query
RETRIEVAL_MIN_SCORE	0.35	Relevance floor; below this a chunk is discarded
MAX_CITATIONS	6	Maximum distinct files cited back to the user
CHUNK_SIZE / CHUNK_OVERLAP	1000 / 200	Splitter geometry

8.3 Security

Variable	Default	Notes
API_KEY	(empty)	Empty disables auth entirely. Set this when hosted.
PROTECT_CHAT	false	Require the key for /chat as well
ALLOW_LOCAL_INGEST	true	Set false when hosted
LOCAL_INGEST_ROOT	(empty)	Optional containment directory
CORS_ORIGINS	localhost:3000	Comma separated, exact scheme and host
CORS_ORIGIN_REGEX	(empty)	Full-match pattern for preview hostnames

8.4 Storage

Variable	Default	Notes
CHROMA_PERSIST_DIR	./chroma_data	Point at a mounted disk when hosted
UPLOAD_DIR	./uploads	Point at a mounted disk when hosted
CHROMA_HOST / CHROMA_PORT	localhost / 8001	Leave unset to force the embedded client
MONGODB_DB_NAME	copilot_rag	Applied separately from the URL

9. Deployment

9.1 Database

Render offers no managed MongoDB, so the database is a MongoDB Atlas M0 cluster. Network access must permit the API's egress addresses; on instance types without static outbound IPs this means allowing all addresses and relying on credential strength. Passwords containing reserved URI characters must be percent-encoded.

The dnspython package is pinned explicitly. PyMongo does not require it, and without it every mongodb+srv:// URI raises a configuration error during startup, which the lifespan handler converts into process exit - producing a container that crash-loops with no obvious cause.

9.2 API service

Setting	Value
Root directory	backend
Runtime	Docker
Instance type	Starter or higher (free tiers provide no persistent disk)
Health check path	/health
Disk mount	/app/data, 1 GB

The container binds the port supplied by the platform. Because the exec form of CMD performs no variable substitution, the start command is wrapped in a shell, with a local default so docker-compose continues to work unchanged.

```
CMD ["sh", "-c", "uvicorn app.main:app --host 0.0.0.0 --port ${PORT:-8000}"]
```

9.3 Web application

Variables prefixed NEXT_PUBLIC_ are compiled into the JavaScript bundle at build time rather than read at runtime. Setting the API URL without triggering a rebuild therefore changes nothing - the redeploy is a required step, not an optimisation.

```
echo "https://your-service.onrender.com" \
  | npx vercel env add NEXT_PUBLIC_API_URL production
npx vercel --prod
```

10. Known Limitations

Documented deliberately: each of these is a real constraint of the current system rather than a hypothetical risk.

Local directory ingestion is unavailable when hosted

The endpoint resolves the supplied path against the server's filesystem. On a hosted instance that is the container, not the operator's machine, so the request always fails. GitHub URL and ZIP upload are the supported paths in a deployed environment.

Ephemeral storage silently destroys the index

Without a mounted disk, the vector store and uploaded archives are erased on every restart while MongoDB retains its metadata. The reconciliation described in section 7 makes the resulting state visible, but only a persistent disk prevents it.

Cold starts can exceed the model timeout

Suspended instances take roughly fifty seconds to wake, while the per-call model timeout is forty-five seconds. The first request after an idle period will usually fail on instance types that suspend.

Retrieval is dense-only

There is no lexical (BM25) channel, no reranking stage and no query expansion. Exact-symbol lookups therefore depend entirely on the embedding model placing the symbol near the query in vector space. Hybrid retrieval with a reranking pass is the highest-value next improvement.

Chunking is structure-agnostic

Splitting is performed on character counts, so a chunk boundary can fall in the middle of a function. Language-aware splitting would raise the quality of each retrieved unit.

The relevance floor rests on a small sample

The 0.35 threshold was calibrated against one repository and a pair of contrasting queries. It separates those cases cleanly but should be re-measured on a broader set, and must be re-derived if the embedding model changes.

Roadmap	Rationale
Hybrid retrieval (BM25 + dense) with score fusion	Recovers exact identifier matches that dense search misses
Cross-encoder reranking of candidates	Improves ordering, allowing a smaller and cleaner context window
Language-aware chunking	Keeps functions and classes intact within a single chunk
Structured logging with request identifiers	Makes a single request traceable across nodes and services
Index-aware health endpoint	Surfaces vector-store drift without waiting for a user to hit it

Live application: https://frontend-theta-olive-55.vercel.app/